A Bit-Exact, Processor-Free FPGA Adaptive Noise Canceller: Fair Comparison of LMS, NLMS and Divider-Free NLMS-Shift on Cyclone IV E

Satvik Garg[a], Aditya Gupta[a], Jaffino G[a,*]

[a]*School of Electronics Engineering (SENSE), Vellore Institute of Technology, Vellore, India*

Abstract

Adaptive noise cancellation with a reference input is a classic application of the least-mean-square (LMS) family, and small FPGAs can run it at audio rates without a processor. Published FPGA implementations, however, rarely compare algorithm variants under equal tuning effort, and few show that the hardware reproduces the reference model. This paper presents ULLAP, a parameterisable 12-bit, 32-tap noise canceller for the Terasic DE2-115 (Cyclone IV E) with a WM8731 codec, supporting LMS, NLMS with a restoring divider, and NLMS-Shift, in which the normalisation is a leading-one detection and a barrel shift. All three share one time-multiplexed multiplier and are verified bit-exact against an integer model in 86 regression tests, including the complete codec interface. With step sizes tuned on held-out seeds, NLMS-Shift matches NLMS within the 95% confidence interval on stationary and amplitude-step noise (22.37 and 24.33 dB final noise reduction over 20 seeds) and is about 2 dB better than LMS on amplitude steps, while needing 72 instead of 127 busy cycles per sample and 16% fewer register-transfer-level bit toggles than NLMS. On real recorded noise and speech at 0 dB input SNR, the three algorithms remove 11.7–14.9 dB of noise and raise STOI from 0.69–0.78 to 0.95–0.98 and narrow-band PESQ from 1.4–1.5 to 2.3–2.8, with NLMS 0.4–0.6 dB ahead of LMS and NLMS-Shift between them. A word-length study at the tuned settings shows 11 bits is the shortest word within 3 dB of floating point and 12 bits keeps the loss below 0.4 dB

*Corresponding author. [TBD: e-mail and author order to be confirmed by the authors]

on broadband noise. All variants close timing at 50 MHz. [TBD: One sentence with the measured on-board noise reduction and the bit-exact capture result.]

Keywords: adaptive noise cancellation, NLMS, divider-free normalisation, FPGA, fixed-point arithmetic, bit-exact verification

---

1. Introduction

Adaptive noise cancellation (ANC) estimates the noise that reaches a primary sensor from a separate reference sensor and subtracts it [1]. The LMS algorithm and its normalised form (NLMS) remain the standard choice for this task because of their low cost and robustness. NLMS makes the adaptation speed independent of the input level, but it needs a division per sample, which is expensive in hardware. Replacing that division by a power-of-two approximation of the input power is an old idea [2], and quantised-normaliser and lookup-table reciprocals have been proposed since [3, 14].

Two questions are left open by the FPGA literature on these filters. First, variants are usually compared with different tuning effort, so a reported advantage can come from an under-tuned baseline; this happened in an earlier version of this work, where a 5.7 dB advantage of the divider-free variant over NLMS disappeared after NLMS was regularised fairly. Second, hardware results are rarely tied to the reference model: resource and timing reports show that a design fits, not that it computes what the simulation claims.

This paper addresses both on a low-cost platform and makes the following contributions:

1. A single serial core that implements LMS, NLMS (restoring divider) and NLMS-Shift with one shared multiplier, so the three are compared on identical datapath, word length and filter length.
2. A fair comparison: every algorithm's step size and regulariser are chosen on held-out seeds with the same criterion, and results are reported with 20-seed 95% confidence intervals, on synthetic scenarios and on real recorded noise and speech.
3. A bit-exact verification chain from a Python integer model to the RTL and the board wrapper (86 regression tests including codec, freeze and reset behaviour), [TBD: and an on-board capture showing the silicon reproduces the model sample for sample].

4. A reproducible word-length study and a complete processor-free audio system that closes timing at 50 MHz on Cyclone IV E, released with all scripts and data.

The algorithmic idea behind NLMS-Shift is not claimed as new; Section 2 positions it.

2. Related work

[TBD: Authors: read each cited paper in full and check the numbers; docs/related_work.md lists what was verified.] Garrett et al. [2] replaced the NLMS division in a decision-feedback equaliser by power-of-two operations obtained from leading-zero counts of the error and the filter energy, and cite earlier real-valued power-of-two NLMS designs; NLMS-Shift is the real-valued, error-exact special case of that approach. Oba et al. [4] quantised the normaliser of an NLMS adaptive-array processor to one bit so that the division became a shift, as described in [7]. Zhao et al. [3] quantised the normaliser of a variable-regularised LMS beamformer to a few levels implemented with canonical signed digits and report at least 21% fewer LUTs and 26.85% higher throughput than a divider design on Virtex-7 (22-bit words, eight sensors). Yao and Huang [5] approximated the reciprocal of the divisor with CSD digits. Huang et al. [6] replaced it by a 6-bit-input, 4-bit-output table; on the same DE2-115 board used here, their 16-tap, 16-bit fully parallel NLMS FIR filter uses 11,276 logic elements, runs at 91 MHz with 11 cycles per sample, and reaches 10.52 nJ per sample against 18.60 nJ for a 4-bit SRT divider. Huang and Yao [7] proposed a binary reciprocal approximation built from a leading-one detector and a barrel shifter for a fifth-order NLMS IIR filter, again on the DE2-115, and report 5.07 nJ per sample against 8.33 nJ for a 15-bit SRT divider. Lee [14] describes the table method in a thesis. Jagadeeswari et al. [15] retimed an NLMS audio filter on Spartan-6 to 74.67 MHz. Fohl et al. [16] built an LMS noise canceller on Spartan-3 with an audio codec and measured it with a dummy head. The base work of this project [17] simulated an LMS feedback suppressor in MATLAB and Vivado without numeric results. Pathi et al. [8] implemented NLMS on an FPGA with a leading-one detector, priority encoder and barrel shifter that performs the division in one cycle, and report about 50% fewer LUTs and 30% lower latency than a conventional NLMS with little effect on convergence; this is the normaliser NLMS-Shift uses, so it is closely related prior

work. Karakaya and Aytar [9] compared NLMS and variable-step-size LMS on a Xilinx XC7A100T with 16-bit arithmetic for underwater ping detection, raising the SNR from 8.85 dB to 29.36 dB (NLMS) and 28.57 dB (VSLMS) with under 3% of the device's LUTs. On the architecture side, Meher and Park [10] reduced the adaptation delay and area of fixed-point LMS filters and report about 17% lower area-delay product for 8 to 32 taps. For acoustic noise control, Shi et al. [11] proposed a systolic FxLMS structure for FPGA headsets, Klemd et al. [12] a multi-channel feedback FxLMS architecture validated on a 2×2 active headrest, and Park and Park [13] a low-power FPGA noise canceller that replaces LMS by CNN-selected fixed filters. [TBD: Pathi et al. [8] was read at abstract level only; check its device, word length and taps in the full text before submission. Oba et al. is cited from the description in [7].] None of these works compares LMS, a true divider and a shift normaliser on one core with equal tuning and statistical tests, and few test a complete audio system with a codec on a board. Compared with these, ULLAP's contribution is the equal-effort three-way comparison on one core, the bit-exact chain to hardware, and the evaluation on real recordings.

3. Algorithms and fixed-point model

With reference vector $\mathbf{x}[n] = [x[n], \ldots, x[n-L+1]]^T$, weights $\mathbf{w}[n]$ and primary input $d[n]$, all three algorithms compute

$$y[n] = \mathbf{w}^T[n]\mathbf{x}[n], \qquad e[n] = d[n] - y[n], \qquad \mathbf{w}[n+1] = \mathbf{w}[n] + \mu[n]\, e[n]\, \mathbf{x}[n], \tag{1}$$

with $\mu[n] = \mu$ for LMS and $\mu[n] = \mu/(P[n] + \epsilon)$, $P[n] = \|\mathbf{x}[n]\|^2$, for NLMS. NLMS-Shift replaces the division by a shift:

$$\mu[n] = 2^{-(\lfloor \log_2(P[n]+\epsilon) \rfloor + 1 + K)}, \tag{2}$$

so that $\mu/(2(P+\epsilon)) < \mu[n] \le \mu/(P+\epsilon)$ with $\mu = 2^{-K}$: the effective step lies within a factor of two below the NLMS step, which keeps the NLMS stability bound. In hardware the exponent is the position of the leading one of $P[n] + \epsilon$, found by a priority encoder, and the update is a barrel shift of $e[n]x[n-i]$.

All signals and weights are signed $Q1.(W-1)$ with $W = 12$; the accumulator has $2W + \lceil \log_2 L \rceil$ bits; every narrowing point saturates and raises a sticky flag; rounding is to nearest. The power $P[n]$ is updated incrementally by adding $x^2[n]$ and subtracting $x^2[n-L]$. The integer model

(`py/fixedpoint.py`, `py/rtl_model.py`) mirrors these widths exactly and generates the golden vectors.

4. Hardware architecture

4.1. Core

The core (`rtl/anc_core.v`, vendor-neutral Verilog) processes one sample per `in_valid` strobe with a single multiplier pipelined in three stages: L cycles for the filter sum, then the error, then L cycles of weight updates. NLMS adds a sequential restoring divider for the gain; NLMS-Shift replaces it by a leading-one detector and a shift-amount register. A `freeze` input holds the weights while filtering continues. The input-to-output latency is $L + 3$ cycles for LMS and $L + 5$ for NLMS and NLMS-Shift (35 and 37 cycles at $L = 32$), and the core is busy for 73, 127 and 72 cycles per sample respectively.

4.2. Board wrapper

The DE2-115 wrapper contains an I^2C engine that configures the WM8731 at reset (11 registers, acknowledge-checked), an I^2S receiver and transmitter clocked by the 50 MHz system clock with the codec's bit and word clocks sampled as data, two-flop synchronisers on the reset and switches, a frame-latched bypass multiplexer, an overrun flag, and four registered test points on the GPIO header. The codec runs as master from a 12.5 MHz clock with the sample-rate register set to 1536 fs, giving 8.138 kHz, the rate of the simulation study. No processor or vendor IP is used.

5. Verification

Table 1 summarises the verification. The core testbench compares every $y[n]$ and $e[n]$, the final weights, the sticky flags and the latency with golden vectors for 68 configurations (W from 8 to 31, L from 2 to 32, saturation corner cases, freeze patterns). The top-level testbench uses a behavioural WM8731 model that checks every I^2C write and follows the programmed sample rate, and runs tests T1–T10: configuration, NACK handling, bypass, end-to-end cancellation against golden vectors, freeze, reset, flags, clocks, a 45-point skew sweep, the 8 kHz rate and the GPIO timing. Mutation tests confirm the testbenches fail on injected faults. Verilator `-Wall` lint leaves no unwaived warning.

Table 1: Verification summary (`results/metrics/iverilog_regression_summary.txt`).

Suite	Tests	Result
Core golden vectors (`tb_anc_core`)	68	all pass, bit-exact
Protocol corner cases	4	pass
Freeze hold/toggle variants	3	pass
Top level with codec model (T1–T10)	11	pass
Python unit tests	46	pass
Verilator lint (`-Wall`)	–	0 unwaived
On-board Signal Tap capture (T9)	–	[TBD: board]

6. Experimental set-up

Synthetic scenarios follow the reference model $x = v$, $d = s + h * v + n_s$ at 8 kHz with 16,000 samples: stationary coloured noise, amplitude steps ($\times 3$ then $\times 0.25$), an abrupt path switch at the midpoint, and a recorded chord as the noise source. Step sizes and regularisers were selected on seeds 101–103 by maximising the mean final noise reduction over stationary, amplitude-step and path-switch scenarios with zero overflow: LMS $\mu = 0.25$; NLMS $\mu = 0.25$, $\epsilon = 1$; NLMS-Shift $K = 3$, $\epsilon = 0.25$. Results are reported on seeds 1–20. The final noise reduction is $\mathrm{NR} = 10 \log_{10} \sum(d - s)^2 / \sum(e - s)^2$ over the last 25% of samples.

For real recordings, four noise files (air conditioner, vacuum cleaner, copy machine, babble) and ten utterances from MS-SNSD [18] were resampled to 8 kHz; each seed draws a noise segment, an utterance and a random 16-tap acoustic path, at 0 dB input SNR, with and without speech leakage into the reference at -20 dB. Noise reduction, STOI [19] and narrow-band PESQ [20] are computed after a 2 s convergence window.

7. Results

7.1. Synthetic scenarios

Table 2 shows that NLMS-Shift ties NLMS on stationary noise and amplitude steps and ties LMS on stationary noise and the path switch. Both normalised algorithms are about 2 dB better than LMS on amplitude steps, where LMS's fixed step is either too slow after the drop or too noisy before it: no LMS step size reaches 22 dB on stationary noise and 24 dB on amplitude steps at once, while both NLMS variants do (Fig. 1). On the recorded chord

Table 2: Final noise reduction, dB, mean [95% CI] over seeds 1–20, $W = 12$, $L = 32$ (`results/paper_numbers.csv`).

Scenario	LMS	NLMS	NLMS-Shift
Stationary	22.39 [22.12, 22.66]	22.28 [22.01, 22.54]	22.37 [22.11, 22.63]
Amplitude steps	22.36 [21.72, 23.00]	24.16 [23.58, 24.73]	24.33 [23.88, 24.78]
Path switch	18.95 [18.81, 19.09]	18.53 [18.39, 18.67]	18.91 [18.76, 19.06]
Recorded chord	5.69 [5.66, 5.73]	5.73 [5.69, 5.76]	5.46 [5.42, 5.50]

Table 3: Paired differences in final NR over seeds 1–20 (`results/metrics/paired_stats.csv`).

Scenario	Pair	Mean diff. (dB)	95% CI	p (paired t)	TOST ±0.5 dB
Stationary	NLMS-Shift − NLMS	+0.10	[+0.01, +0.18]	0.028	equivalent
Stationary	NLMS-Shift − LMS	-0.02	[-0.07, +0.04]	0.58	equivalent
Stationary	NLMS − LMS	-0.11	[-0.18, -0.04]	0.0038	equivalent
Amplitude steps	NLMS-Shift − NLMS	+0.17	[-0.12, +0.47]	0.23	equivalent
Amplitude steps	NLMS-Shift − LMS	+1.97	[+1.51, +2.43]	2.8e-08	not equiv.
Amplitude steps	NLMS − LMS	+1.79	[+1.38, +2.21]	2.5e-08	not equiv.
Path switch	NLMS-Shift − NLMS	+0.38	[+0.30, +0.47]	1.9e-08	equivalent
Path switch	NLMS-Shift − LMS	-0.04	[-0.07, -0.01]	0.018	equivalent
Path switch	NLMS − LMS	-0.42	[-0.50, -0.34]	1.1e-09	equivalent
Recorded chord	NLMS-Shift − NLMS	-0.27	[-0.30, -0.24]	7.7e-14	equivalent
Recorded chord	NLMS-Shift − LMS	-0.24	[-0.26, -0.22]	2.6e-16	equivalent
Recorded chord	NLMS − LMS	+0.03	[+0.00, +0.06]	0.036	equivalent

NLMS-Shift is 0.2–0.3 dB worse. Paired per-seed tests (Table 3) make "tie" precise: every NLMS-Shift versus NLMS difference is inside a ±0.5 dB equivalence margin (TOST, $p < 0.05$), although some are statistically non-zero, for example +0.38 dB on the path switch; the 1.8–2.0 dB advantage of the normalised algorithms over LMS on amplitude steps is significant ($p < 10^{-7}$).

Figures 2–7 show single runs (seed 1) at the fair-tuned settings. Fig. 2 gives the running noise reduction in every scenario, including a closed-loop feedback case in which all three algorithms settle near 4 dB; Fig. 3 and Fig. 4 show the signals and the residual error power for stationary noise; Fig. 5 shows the response to the amplitude steps, where LMS loses about 3 dB after the drop while both normalised algorithms recover; Fig. 6 compares final and whole-run noise reduction; and Fig. 7 shows that the coefficient trajectories of the three algorithms follow the same path.

7.2. Real noise and speech

Table 4 shows that on all four real noises the filter roughly doubles PESQ and brings STOI close to 1, and that the order of the algorithms is the same

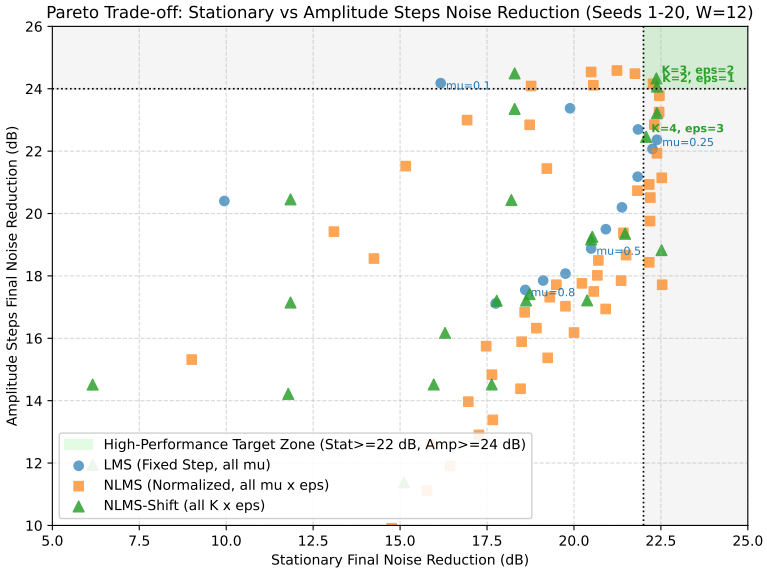


Figure 1: Stationary versus amplitude-step noise reduction across step sizes (seeds 1–20).

everywhere: NLMS 0.4–0.6 dB ahead of LMS, NLMS-Shift 0.15–0.35 dB behind NLMS. Real noise is less predictable than the coloured synthetic noise, so the noise reduction is 8–11 dB lower than in Table 2. When −20 dB of the speech leaks into the reference, the filter also partly cancels speech: noise reduction drops by 2.3–4.6 dB, PESQ by 0.25–0.6, and saturation events appear (18,500 over 19.2 million processed samples, most for the air conditioner), so reference-microphone placement matters more than the choice of algorithm (`paper/tab_real_audio_xt.tex`).

7.3. Word length

At the tuned settings, the worst-scenario loss relative to the floating-point run of the same algorithm is 4.1–5.9 dB at $W = 10$, 0.9–1.2 dB at $W = 11$ and 0.3–0.4 dB at $W = 12$ (Fig. 8); 11 bits is the shortest word within the 3 dB criterion and 12 bits was kept for margin. On the tonal chord recording the fixed-point filter outperforms floating point by up to 3.4 dB at $W = 11$ and loses 1.3 dB at $W = 13$: on this narrow-band signal quantisation changes the effective adaptation, so the chord result is sensitive to the word length and should not decide it.

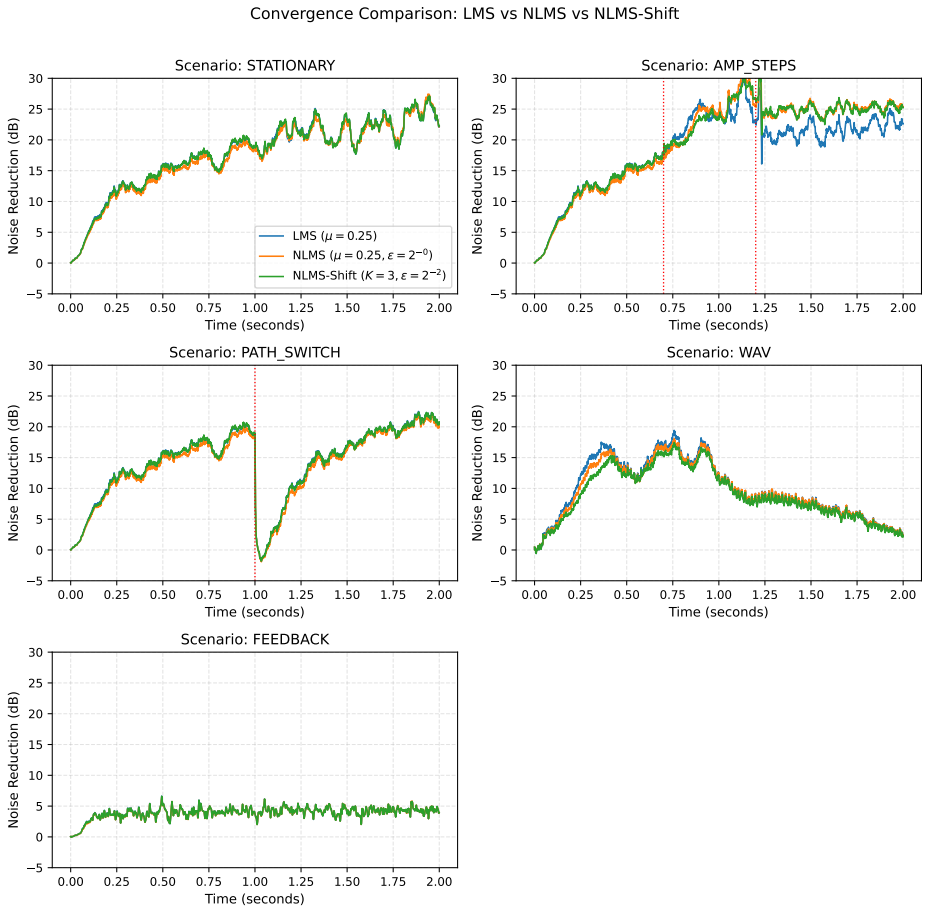


Figure 2: Running noise reduction for all scenarios (seed 1, $W = 12$, $L = 32$, fair-tuned settings). Dotted lines mark the amplitude steps and the path switch.

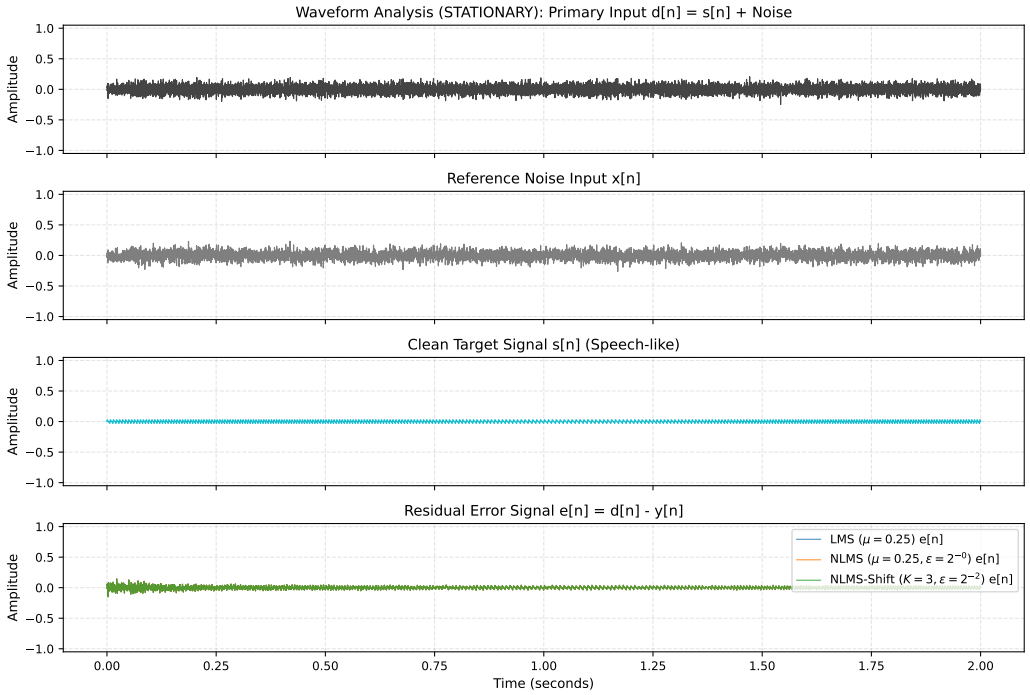


Figure 3: Primary $d[n]$, reference $x[n]$, clean signal $s[n]$ and residual $e[n]$ of each algorithm, stationary scenario (seed 1).

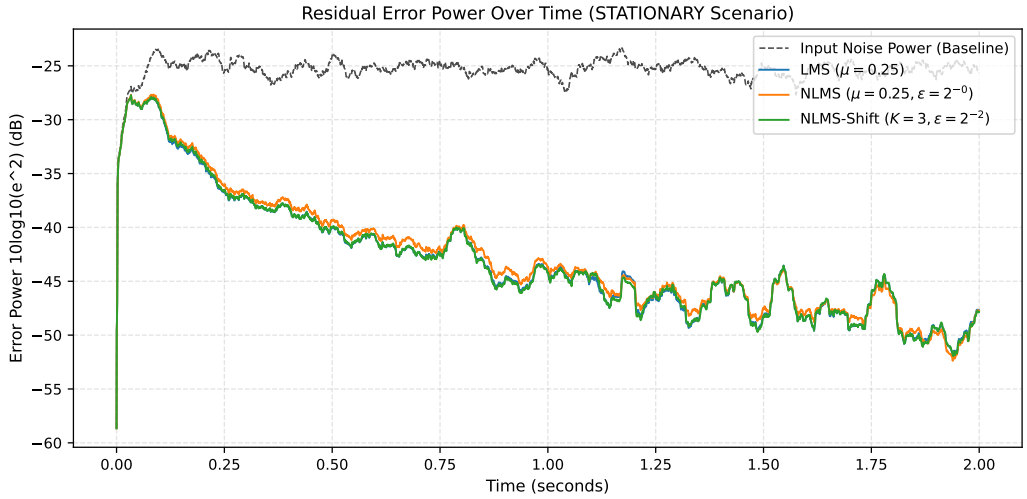


Figure 4: Residual error power over time against the input noise power, stationary scenario (seed 1).

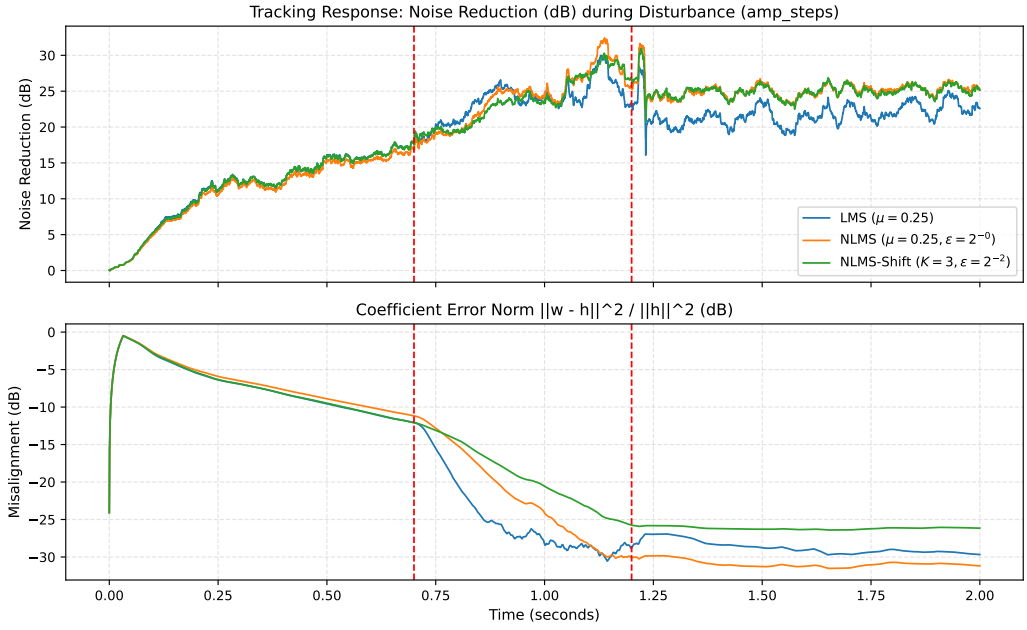


Figure 5: Tracking of the amplitude-step disturbance (seed 1): noise reduction and coefficient misalignment.

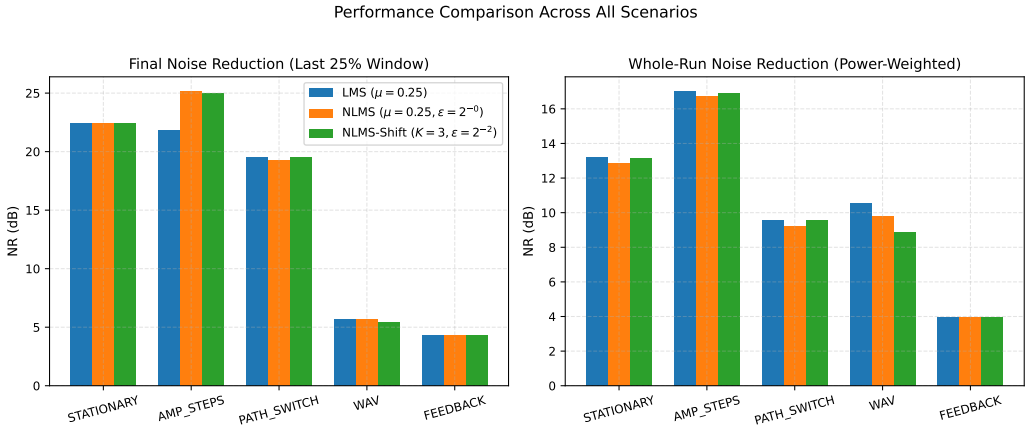


Figure 6: Final (last 25%) and whole-run noise reduction per scenario (seed 1). The 20-seed means are in Table 2.

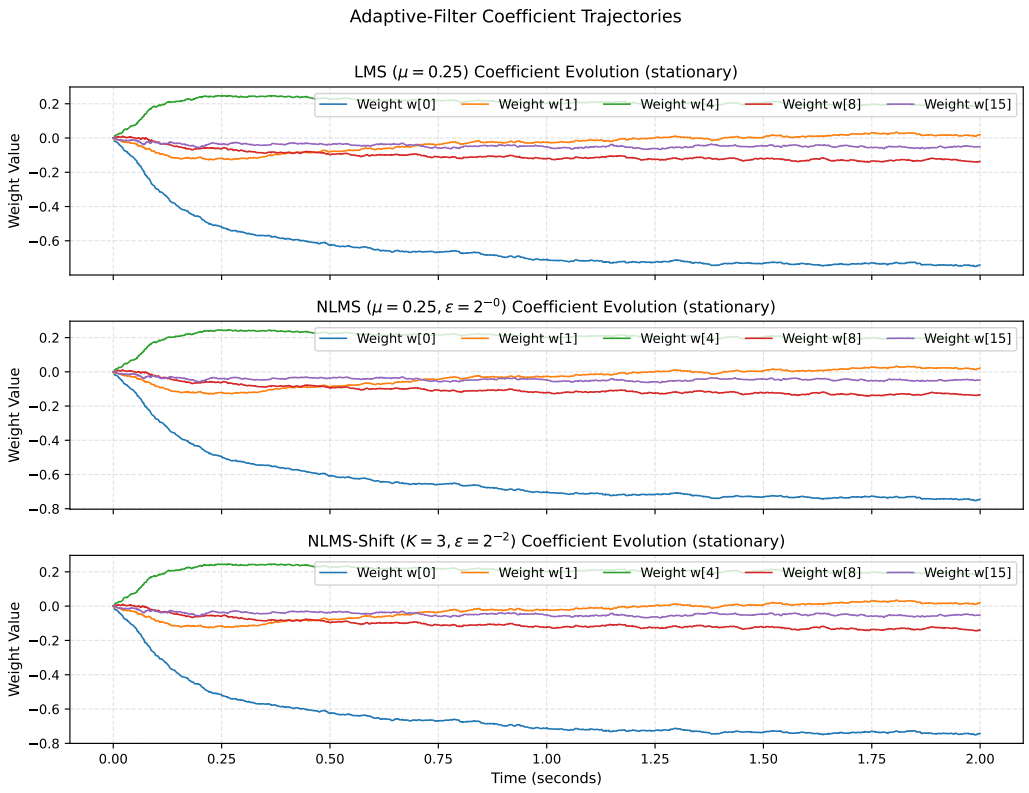


Figure 7: Trajectories of representative taps $w_0, w_1, w_4, w_8, w_{15}$ (stationary scenario, seed 1).

Table 4: Real recorded noise and speech (MS-SNSD), fixed point $W = 12$, mean ± 95% CI over 20 seeds, no speech leakage into the reference; STOI and PESQ of e against s, with the unprocessed input in brackets (`results/metrics/real_audio_summary.csv`).

Noise	Metric	LMS	NLMS	NLMS-Shift
Air conditioner	NR (dB)	11.65 ± 0.75	12.04 ± 0.82	11.89 ± 0.80
	STOI (input 0.784)	0.948	0.947	0.948
	PESQ (input 1.54)	2.34	2.30	2.33
Vacuum cleaner	NR (dB)	14.32 ± 0.32	14.90 ± 0.33	14.56 ± 0.35
	STOI (input 0.730)	0.982	0.983	0.982
	PESQ (input 1.39)	2.81	2.81	2.81
Copy machine	NR (dB)	14.02 ± 0.38	14.63 ± 0.34	14.39 ± 0.32
	STOI (input 0.694)	0.977	0.978	0.977
	PESQ (input 1.36)	2.74	2.75	2.73
Babble	NR (dB)	11.95 ± 0.42	12.43 ± 0.49	12.27 ± 0.46
	STOI (input 0.719)	0.956	0.955	0.955
	PESQ (input 1.48)	2.48	2.47	2.47

7.4. Hardware cost

LMS is about 30% smaller than either normalised variant. NLMS-Shift and NLMS use about the same logic, because the divider's area is replaced by the leading-one detector, shifter and wider update path, but NLMS-Shift needs 43% fewer busy cycles per sample and toggles 16% fewer bits per sample, a proxy for dynamic energy. At 8.138 kHz the busiest variant uses 2% of the 6,144-cycle sample period, so a single core could serve many channels or much longer filters. The complete board design with synchronisers (NLMS-Shift) uses 2,627 logic elements and closes at 60.36 MHz with +3.432 ns setup slack. For comparison with [6, 7], which use the same board, energy per sample should be reported as dynamic power divided by the sample rate, from the Quartus Power Analyzer. Their designs are fully parallel and run at megasamples per second, while ULLAP shares one multiplier and runs at 8.138 kHz, so area and energy are compared per tap and per sample, not as raw totals.

Figures 9–12 show the same build data graphically: logic and register use (Fig. 9), $F_{\max}$ and setup slack at the slow 85 °C corner (Fig. 10), noise reduction against logic cost (Fig. 11), and latency and busy time against the 122.88 μs sample period (Fig. 12).

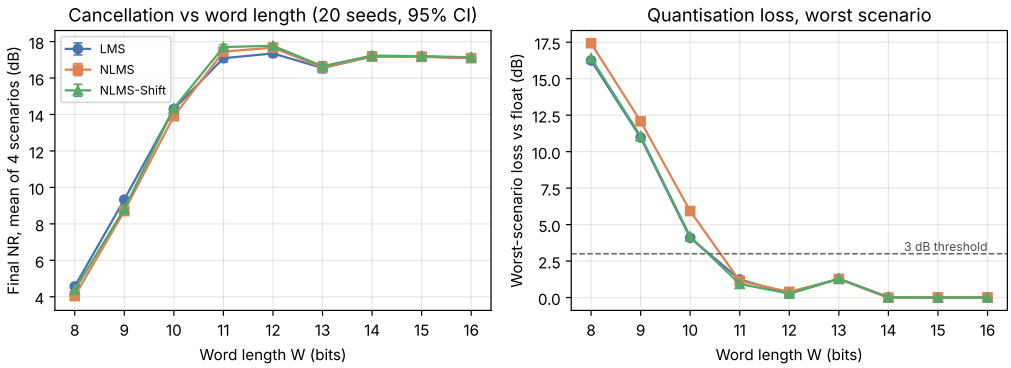


Figure 8: Noise reduction and worst-scenario quantisation loss versus word length (20 seeds).

Table 5: Cyclone IV E EP4CE115F29C7, $W = 12$, $L = 32$, Quartus Prime Lite 25.1 (`results/quartus_reports/`, `results/metrics/power_activity.csv`).

	LMS	NLMS	NLMS-Shift
Logic elements	1,885	2,697	2,690
Registers	1,148	1,425	1,213
9-bit multipliers	4	4	4
$F_{\max}$, slow 85 °C (MHz)	59.90	61.72	61.98
Latency (cycles)	35	37	37
Busy cycles per sample	73	127	72
RTL bit toggles per sample	5,668	7,146	6,012
Dynamic power (mW)	[TBD:]	[TBD:]	[TBD:]

[TBD: Rebuild all three variants on the final commit and replace this table with that build; add Power Analyzer results (docs/power_flow.md).]

7.5. On-board results

[TBD: Board tests T1–T11 (docs/board_test.md): bit-exact capture result, measured latency on the GPIO pins (expected 760 ns between in_valid and out_valid), measured noise reduction per scenario and algorithm next to the simulated value, and the 10-minute soak.]

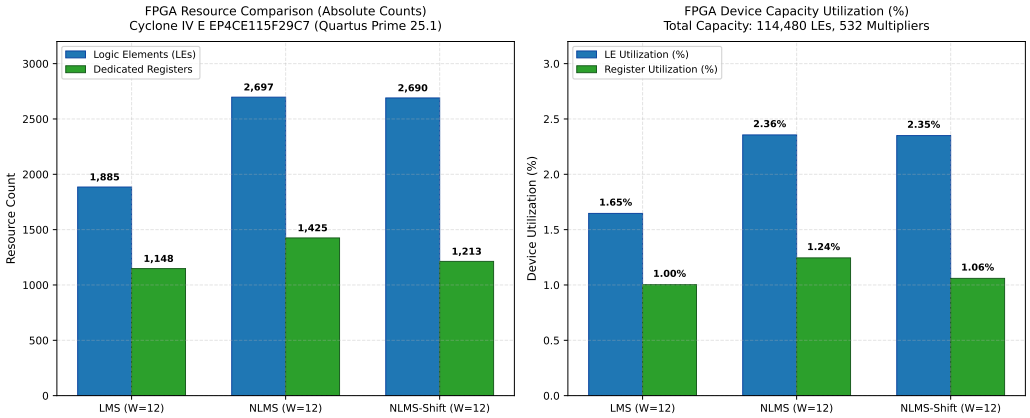


Figure 9: Logic elements and dedicated registers of the three cores on Cyclone IV E EP4CE115 (Quartus Prime Lite 25.1).

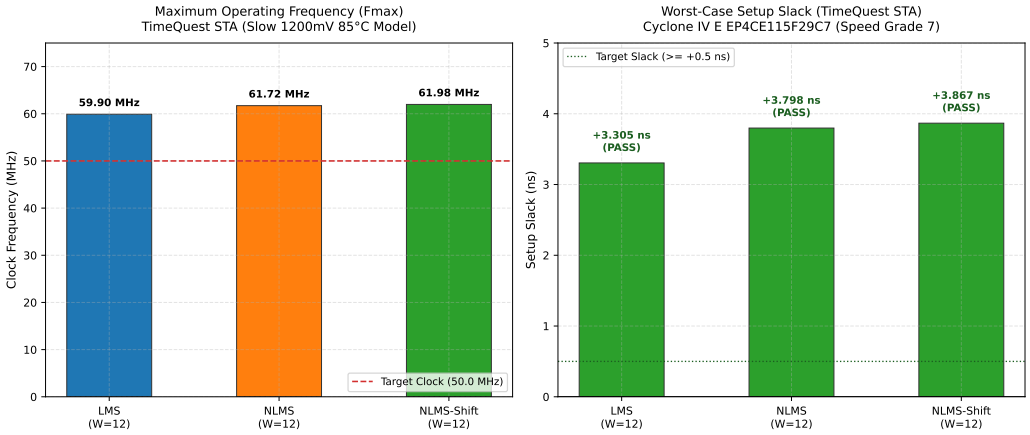


Figure 10: $F_{\max}$ and worst-case setup slack, slow 1200 mV 85 °C model, 50 MHz constraint.

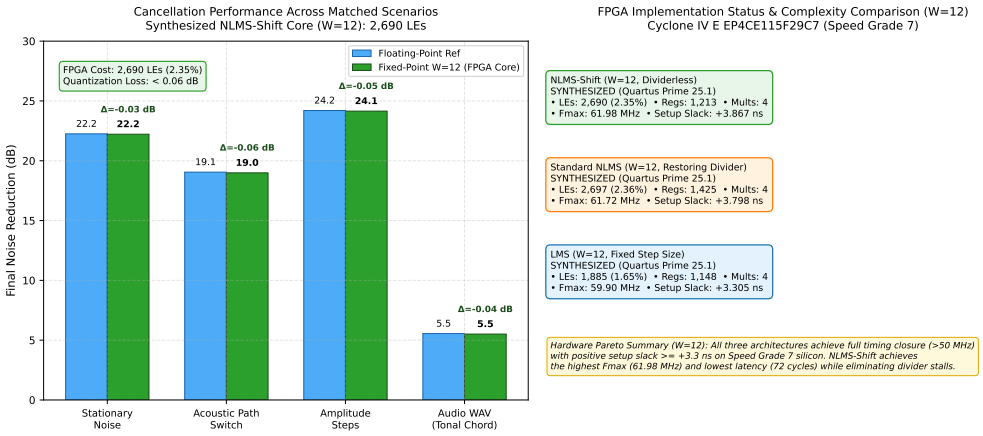


Figure 11: Final noise reduction of the floating-point and $W = 12$ fixed-point NLMS-Shift (seed 1) next to the synthesis results of the three cores.

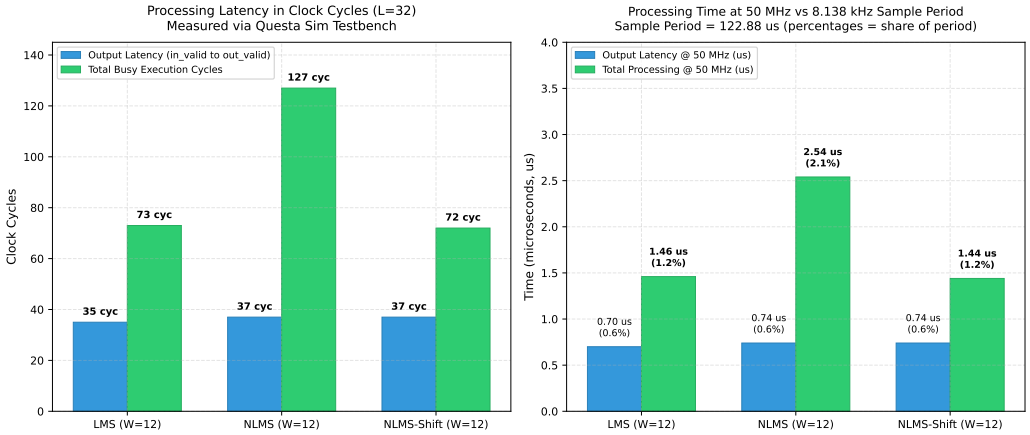


Figure 12: Output latency and busy cycles per sample, and the corresponding time at 50 MHz as a share of the 8.138 kHz sample period.

8. Discussion and limitations

The comparison supports a narrow conclusion: the divider-free normalisation keeps NLMS's level-independence at about LMS's cycle count, so it is the better choice when the noise level changes and throughput or energy per sample matters, while LMS remains the smallest when the level is steady. It does not show that NLMS-Shift is better than NLMS in cancellation. The acoustic paths are simulated, also for the real recordings; the evaluation does not cover acoustic feedback (the closed-loop howling scenario of the base work), for which reference-based ANC is not the right structure. Power is a switching-activity proxy until gate-level analysis is done. [TBD: Update after the board test.]

9. Conclusion

[TBD: Write after the board results: three to four sentences restating the measured outcome.]

Data and code availability

All RTL, testbenches, scripts and result files are in the project repository [TBD: public URL and Zenodo DOI of the tagged release]. The real-audio study fetches its MS-SNSD files from a pinned commit and checks them by SHA-256.

Declaration of generative AI use

[TBD: Authors: describe the AI tools used for code, analysis, literature search and writing, as required by the journal, and state that the authors reviewed and take responsibility for the content.]

References

[1] B. Widrow et al., Adaptive noise cancelling: principles and applications, Proc. IEEE 63 (12) (1975) 1692–1716.

[2] D. Garrett, C. Nicol, A. Blanksby, C. Howland, A low power normalized-LMS decision feedback equalizer for a wireless packet modem, in: Proc. ISLPED, 2002.

[3] W. Zhao, J.Q. Lin, S.C. Chan, H.K.-H. So, A division-free and variable-regularized LMS-based generalized sidelobe canceller for adaptive beamforming and its efficient hardware realization, IEEE Access 6 (2018) 64470–64485.

[4] H. Oba, M. Kim, H. Arai, FPGA implementation of LMS and N-LMS processor for adaptive array applications, in: Proc. ISPACS, Yonago, 2006, pp. 485–488.

[5] C.-Y. Yao, Y.-Z. Huang, The design of the NLMS adaptive filters using the fast-division approximation with CSD encoded divisors, IEEE Trans. Circuits Syst. II: Exp. Briefs 71 (4) (2024) 2459–2463.

[6] Y.-Z. Huang, C.-Y. Yao, P.-Y. Lee, Speeding up the throughput of an NLMS adaptive filter, in: Proc. IEEE ICCE-Taiwan, 2024, pp. 221–222.

[7] Y.-Z. Huang, C.-Y. Yao, Enhancing the power efficiency of the NLMS adaptive IIR filter with fast binary reciprocal approximation, Circuits Syst. Signal Process. 45 (2026) 7561–7584.

[8] A.M.V. Pathi, V. Santosh Kumar, R. Jnaneswari, P. Leela Krishna, P. Swamulu, FPGA implementation of NLMS adaptive filter using shift based division, in: Proc. 7th Int. Conf. Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), Tirunelveli, 2026, pp. 564–569, doi:10.1109/ICICV68925.2026.11554859.

[9] E. Karakaya, O. Aytar, FPGA implementation of NLMS and VSLMS algorithms for underwater acoustics applications, IEEE Access 14 (2026) 53860–53872, doi:10.1109/ACCESS.2026.3681282.

[10] P.K. Meher, S.Y. Park, Area-delay-power efficient fixed-point LMS adaptive filter with low adaptation-delay, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 22 (2) (2014) 362–371, doi:10.1109/TVLSI.2013.2239321.

[11] D. Shi, C. Shi, W.-S. Gan, A systolic FxLMS structure for implementation of feedforward active noise control on FPGA, in: Proc. APSIPA ASC, Jeju, 2016, doi:10.1109/APSIPA.2016.7820755.

[12] A. Klemd, B. Klauer, J. Timmermann, D. Sachau, A flexible multi-channel feedback FxLMS architecture for FPGA platforms, in: Proc.

31st Int. Conf. Field-Programmable Logic and Applications (FPL), 2021, pp. 319–326, doi:10.1109/FPL53798.2021.00063.

[13] S. Park, D. Park, Low-power FPGA realization of lightweight active noise cancellation with CNN noise classification, Electronics 12 (11) (2023) 2511, doi:10.3390/electronics12112511.

[14] P.-Y. Lee, A dividerless NLMS adaptive filter design with the lookup-table method, M.Sc. thesis, National Taiwan University, 2024.

[15] M. Jagadeeswari, P. Rishabavarthani, S. Kumarganesh, S. Kannadhasan, Enhanced adaptive filtering in audio systems: normalized least mean square with cut-set retiming, Analog Integr. Circuits Signal Process. 126 (2026) 12.

[16] W. Fohl, J. Matthies, B. Schwarz, A FPGA-based adaptive noise cancelling system, in: Proc. DAFx, Como, 2009.

[17] W.S.C. Rodrigo, R.A.P. Buddhika, Real-time FPGA-based audio processing for enhanced acoustic feedback suppression, in: Proc. TIDAC Research Symposium, Colombo, 2024.

[18] C.K.A. Reddy et al., A scalable noisy speech dataset and online subjective test framework, in: Proc. Interspeech, 2019, pp. 1816–1820.

[19] C.H. Taal, R.C. Hendriks, R. Heusdens, J. Jensen, An algorithm for intelligibility prediction of time-frequency weighted noisy speech, IEEE Trans. Audio Speech Lang. Process. 19 (7) (2011) 2125–2136.

[20] ITU-T Recommendation P.862, Perceptual evaluation of speech quality (PESQ), 2001.